Forelesning 2

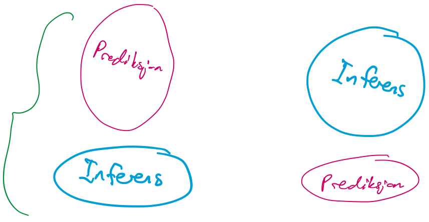
tirsdag 20. januar 2026 12:27

Fra sist forelesning

- Lineær algebra → Data → vektor, matriser
  - → Tensorer
  - → Normer → $\|x\|_p = \left(\sum_i |x_i|^p\right)^{1/p}$
- Kalkulus → Derivasjon → "Finne stigningstall til tangenten"
- Statistikk → Irreducible- og reducible error → $E[(Y-\hat{Y})^2]$
- Maskinlæring → Regresjon vs. klassifikasjon
  - → Prediksjon vs. inferens
  - → Overfitting → modell kan lære støy
  - → Trening, validering og test sett

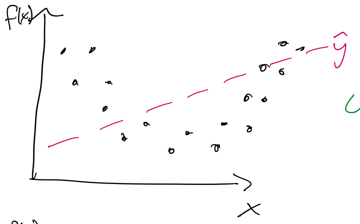
Fokus i kurset

Maskinlæring   Statistikk



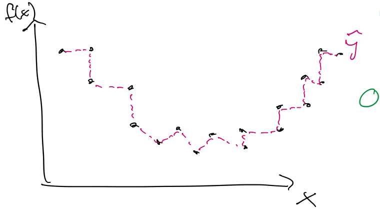
Vi vil være mest her!

Under- og overfitting



Underfitted

Vi ønsker å være mellom disse!



Overfitting

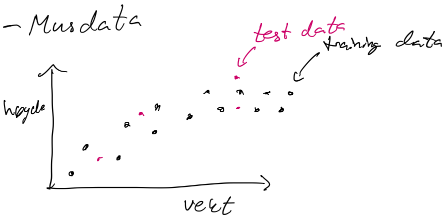
Kan vi oppnå via:

- Regularisering
- Krysvalidering
- Dropout
- Data
- Ensemble

⋮

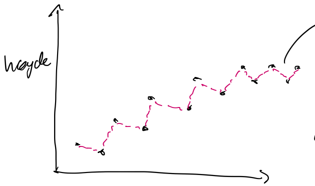
Bias-Variance Tradeoff

- Musdata



- For å lage en modell for dette kan vi bruke lineær regresjon!
  - Men! lineær regresjon har ikke fleksibilitet til å tilpasse seg dataene våre!

→ Lineær regresjon kan altså ikke fange opp det ekte forholdet mellom vekt og høyde.
Denne manglende egenskapen er bias til modellen.



En annen ML-teknikk kan tilpasse seg dataene bedre, fordi den er kjempe fleksibel.
→ Den har lite bias.

Vekt

→ Hvis vi ser på MSE av treningsdataene:
- Vil linear regresjon få en del MSE
- Fleksible teknikker vil få mye lavere MSE.

→ Regner vi ut MSE for test-dataene:
- Da vil fort lineær regresjon vinne!

→ Denne forskjellen i "fits", mellom trening og test sett er en konsekvens av varians.

Statistisk sett, så refereres varians til den andelen prediksjonene vil endre seg, hvis vi "fittes"/tilpasses modellen til et annet treningssett.

Kompleks modell	Enkel modell
* Lav bias.	* Høy bias
* Fleksibel	* Lite fleksibel
* Høy varians	* Lav varians
– Kan være veldig bra – kan fort overfitte!	– Kan ofte gi gode resultat, men fanger sjelden opp kompleksiteten i data.

Vårt ønske:

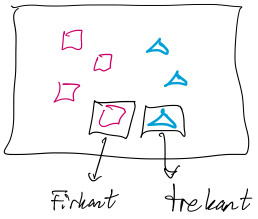
* En modell som har
  - Lav bias → og kan være en modell for det faktiske forholdet mellom features og target.
  - Lav varians → ved å gi konsistente prediksjoner på ulike datasett.

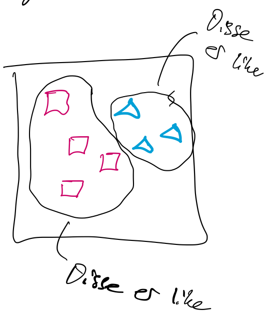
Dette lykkes vi med om vi finner sweet spot mellom en

ENKEL MODELL og KOMPLEKS MODELL

Veiledet læring vs ikke-veiledet læring

Veiledet læring	ikke-veiledet læring
– Merket data	– Umerket data
– Bruker treningsdata	– Bruker all data som input
– Brukes for prediksjon	– Klustering
– Trenger "labels", eller "en fasit"	

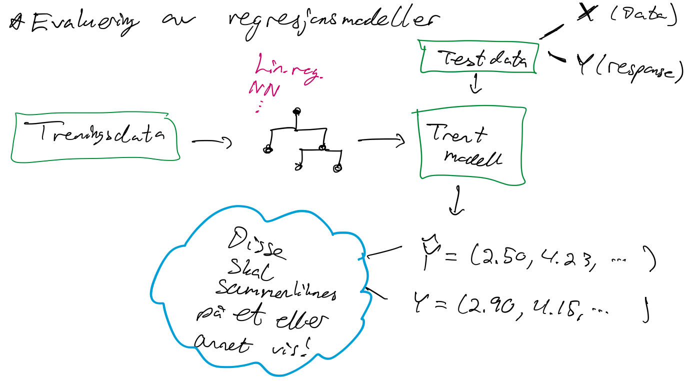




Evaluering av modeller

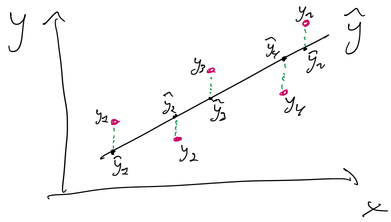
Hvordan vi evaluerer maskinlæringsmodellene våre avhenger av om vi ser på regresjon eller klassifikasjon.

* Evaluering av regresjonsmodeller



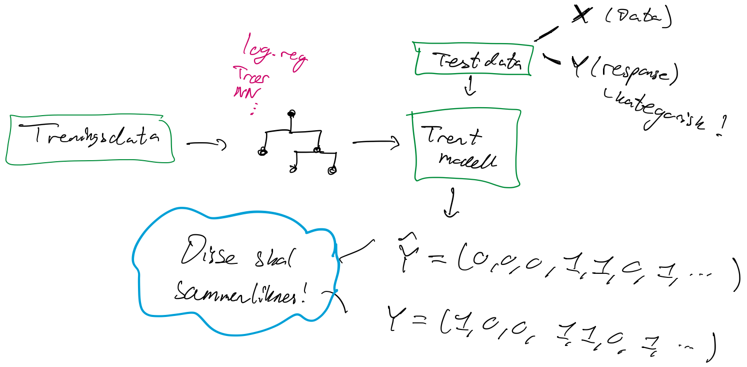
$$\hat{Y} = (2.50, 4.23, \dots)$$

$$Y = (2.90, 4.15, \dots)$$



	MSE	MAE
Formel	$\frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{y}_i)^2$	$\frac{1}{n}\sum_{i=1}^{n}\lvert y_i-\hat{y}_i\rvert$
Fokus	"Straffer" større feil mer	Mindre sensitiv for "outliers"
Tolkning	Verdien er kvadrert, vanskeligere å tolke	Verdien er i samme enhet som målvariabel.
Optimalisering	Lettere å optimalisere matematisk (den deriverte er glatt og kontinuerlig.)	Kan gi ut fordringer (ikke deriverbar i 0) ↳ altså når predikert $\hat{y}_i$ er lik observert y_i

* Evaluering av klassifikasjonsmodeller



$$\hat{Y} = (0,0,0,1,1,0,1,\dots)$$

$$Y = (1,0,0,1,1,0,1,\dots)$$

Obs: Her ser vi på binær (to utfall), kan fint være flere!

Eks: X ~ finansiell / transaksjonsdata

Y ~ responsen $\begin{cases} 0 = \text{ikke-svindel} \\ 1 = \text{svindel} \end{cases}$

Når vi skal sammenlikne prediksjon mot faktisk output – i klassifikasjon trenger vi:

Confusion matrix

Predikert

Observert	0	1
0	True Negative	False Positive
1	False Negative	True Positive

Evalueringsmetrikker

Accuracy: $\frac{TN + TP}{TN + TP + FN + TP}$

(Nøyaktighet)

Andel korrekte pred. av totale datapunkter

Precision: $\frac{TP}{TP + FP}$

"av alle pos. prediksjoner, hvor mange er faktisk positive?"

Recall (sensitivitet): $\frac{TP}{TP + FN}$

"andelen riktig klassifiserte positive, av alle faktiske positive"

F1 -Score: $2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$

"Harmonisk gjennomsnitt av de to over."

Eks:

Binært problem, SVINDEL-DETEKSJON

Data: 0 (ikke svindel): 9900 (samples)
1 (svindel) : 100 samples

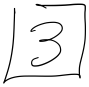
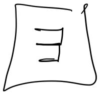
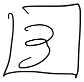
Confusion matrix

Observert	0	1
0	9850	50
1	95	5

Predikert

Ubalanserte datasett får konsekvenser!
Hva skjer her?
Se på de ulike evalueringsmetrikkene!?

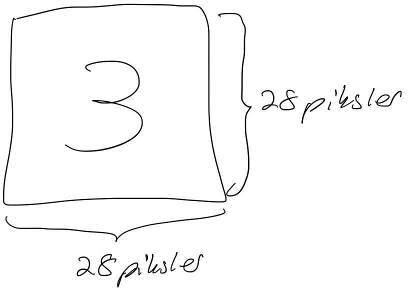
Dyplæring

Hvorfor er  ,  og  alle gjenkjent som tallet tre?

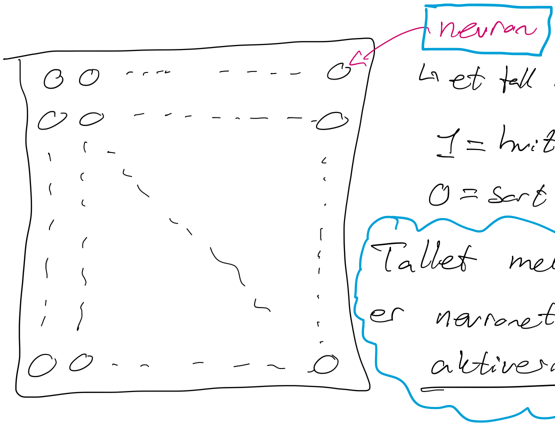
Vi skal lage et program/modell, som kan gjøre dette!

Da skal vi bruke et enkelt nevralt nettverk, kalt

Multi-Layer Perceptron (MLP)



Dette er en matrise av tallet tre:



↳ et tall mellom 0 og 1.

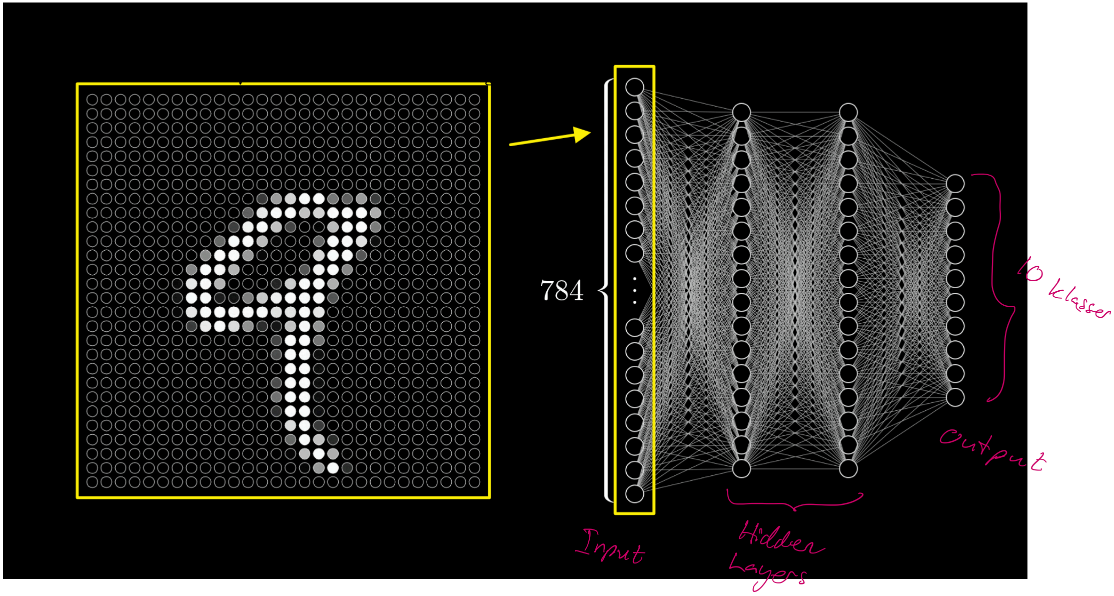
1 = hvit piksel

0 = sort piksel

Tallet mellom 0 og 1 er nevronet sin aktivering

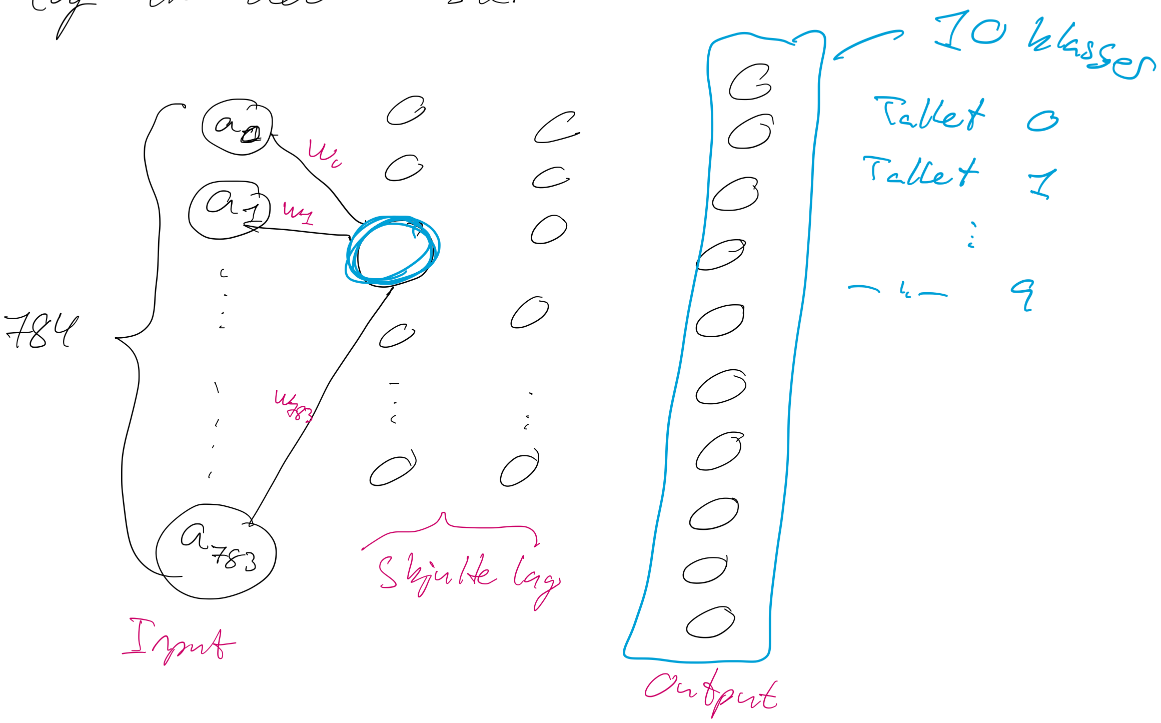
Derfor er analogien til hjernen en ting!

– I hjernen, har nevronene dine enten 1 (aktivert) eller 0 (ikke – aktivert). AI kan ha alt mellom 0 og 1.



- Aktiveringen i ett lag, bestemmer aktiveringen i neste.
  - Alle nevronene er koblet sammen med vekter (w_i) og biaser (b_i)

- Det er slik vi får info fra ett lag til det neste.



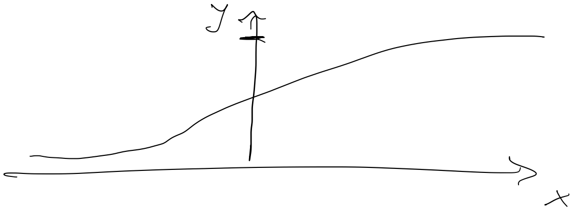
* Hvert nevron har en aktivering (mellom 0 og 1), og vi regner ut en vekted sum.

$$\boxed{w_0 a_0 + w_1 a_1 + \cdots + w_{783} a_{783}}$$

↳ Dette blir et tall mellom $-\infty$ og ∞.

Vi vil ha det tallet mellom 0 og 1!!!

- Derfor har vi aktiveringsfunksjoner!



F.eks. sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

← Dette er en av mange akt. funksjoner!

* Så ved å ta

$$\sigma(w_0 a_0 + w_1 a_1 + \cdots + w_{783} a_{783})$$

"tvinges" denne vekta summen til intervallet $(0,1)$

* MEN!

Kanskje vil vi at en nevron kun blir aktivert hvis den vekta summen er over f.eks. 20

- Da trenger vi en bias for å ta hensyn til dette!

Da blir formelen

bias

vekter

$$\boxed{\sigma(w_0 a_0 + w_1 a_1 + \cdots + w_{783} a_{783} - b)}$$

akt. funksjon

aktivering til nevron

Obs: Dette er kun for ett, enkelt nevron!

Så det er 784 vekter per nevron, pluss hver sin bias!

$$\boxed{\begin{array}{l} 784 \cdot 16 + 16 \cdot 16 + 16 \cdot 10 = 12\,960 \text{ vekter} \\ 16 + 16 + 10 = 42 \text{ biaser} \end{array}}$$

Totalt 13.002 parameter å optimalisere for ett nettverk med to skjulte lag, av 16 nevroner med 10 klasser som output!

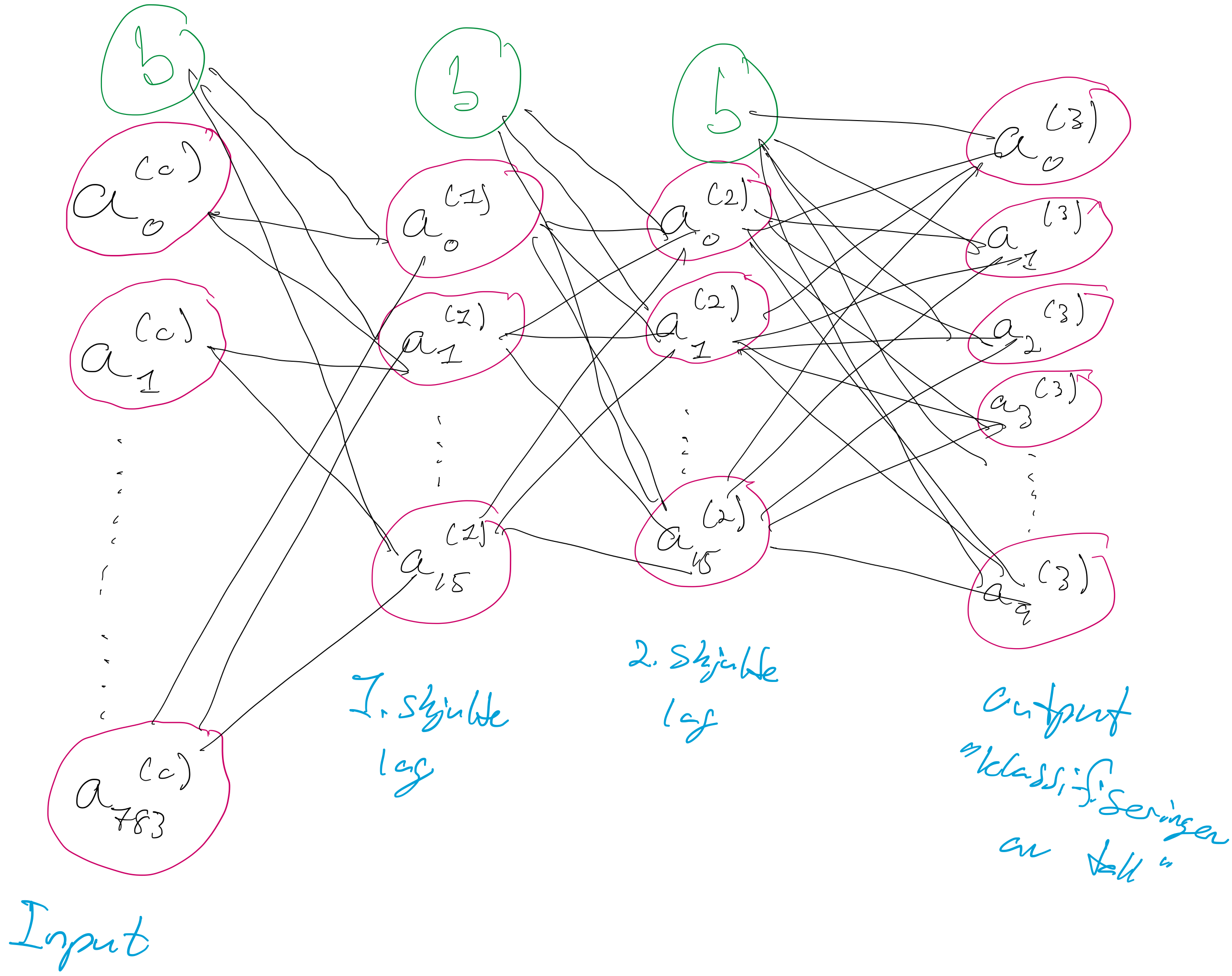
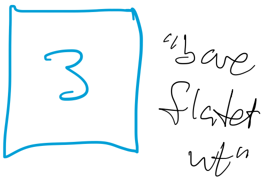
„Læring" betyr at datamaskinen/modellen finner „riktig" eller optimal verdi på disse parameterne!

Vi kan skrive funksjonen matematisk fra input-lag til første skjulte lag, slik:

$$a^{(1)} = \sigma\left(\begin{bmatrix} w_{0,0} & w_{0,1} & \cdots & w_{0,n} \\ w_{1,0} & w_{1,1} & \cdots & w_{1,n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{k,0} & w_{k,1} & \cdots & w_{k,n} \end{bmatrix} \begin{bmatrix} a_0^{(0)} \\ a_1^{(0)} \\ \vdots \\ a_n^{(0)} \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \\ \vdots \\ b_n \end{bmatrix} \right)$$

$$\boxed{a^{(1)} = \sigma\left(W a^{(0)} + b\right)}$$

Hvar:

- $a^{(1)}$ er vektoren av aktiveringen til alle nevronene i første skjulte lag.
- $a^{(0)}$ er vektoren av aktiveringen til nevronene i input-laget.
- W er matrisen med alle vekter mellom $a^{(0)}$ og $a^{(1)}$.
- b er bias-vektoren.